

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Introducción y Contexto

Motivación: NumPy

Primeros Pasos con NumPy

Manipulación de Arrays y Operaciones

Operaciones Numéricas y Práctica

Conclusiones

Introducción y Repaso

- **Sesión anterior** se centró en:
 - Listas y estructuras de datos básicas
 - Bucles for y manipulación de colecciones
- **Conexión:** Hoy aplicaremos estos conceptos para trabajar con arreglos numéricos y NumPy.

- **Comprender** los conceptos básicos de NumPy: arreglos (arrays), vectorización, broadcasting.
- **Practicar** la creación y manipulación de arrays en Google Colab.
- **Aplicar** estas herramientas a ejemplos numéricos y pequeños problemas de Física/Astronomía.
- **Desarrollar** habilidades de trabajo colaborativo y análisis numérico.

- El bucle **for** se utiliza para iterar sobre una secuencia (lista, cadena, etc.).
- Ejemplo básico:

```
1 numeros = [10, 20, 30]
2 for num in numeros:
3     print(num)
```

- En cada iteración, **num** toma un valor de la lista.

Introducción y Repaso $\ni$ Ejemplo: Suma de Elementos de una Lista

```
1 numeros = [1, 2, 3, 4, 5]
2 suma = 0
3
4 for n in numeros:
5     suma += n
6
7 print("La suma es:", suma)
```

- Con **for** iteramos directamente sobre cada elemento.
- Acumulamos la suma en **suma**.
- Resultado: La suma es: 15.

Introducción y Contexto

- Ahora retomamos contenidos para profundizar nuestras habilidades de programación.
- **Objetivo de esta sesión:** Introducir la librería **NumPy** para manejo eficiente de arreglos y cálculos numéricos.

- **Comprender** los conceptos básicos de NumPy: arreglos (arrays), vectorización, broadcasting.
- **Practicar** la creación y manipulación de arrays en Google Colab.
- **Aplicar** estas herramientas a ejemplos numéricos y pequeños problemas de Física/Astronomía.
- **Fomentar** la colaboración y el trabajo práctico en laboratorio.

Motivación: NumPy

Motivación: NumPy $\ni$ ¿Qué es NumPy?

- **NumPy** = **N**umerical **P**ython.
- Ofrece:
 - Estructura central: **ndarray** (arreglo multidimensional).
 - Funciones matemáticas optimizadas para operar sobre estos arreglos.
 - Alto rendimiento (implementaciones en C bajo la interfaz de Python).
- **Base de muchas librerías** científicas (p.e. SciPy, Pandas, etc.).

Motivación: NumPy $\ni$ Ventajas frente a Listas Nativas de Python

- **Vectorización:** escribir operaciones como `arr * 2` en lugar de bucles manuales.
- **Rendimiento:** implementaciones de bajo nivel (C/Fortran) para cálculos numéricos.
- **Herramientas avanzadas:** slicing sofisticado, broadcasting, manipulación de ejes (dimensiones).
- **Compatibilidad** con otras librerías (Matplotlib, SciPy, etc.).

Primeros Pasos con NumPy

Primeros Pasos con NumPy $\ni$ Instalación y Importación

- En muchos entornos (Colab, Anaconda), NumPy ya está incluido.
- Si fuera necesario:

```
pip install numpy
```

- Importación típica:

```
1 import numpy as np
```

- A partir de ahí, usamos `np.array`, `np.mean`, etc.

Primeros Pasos con NumPy $\ni$ Creación de Arrays NumPy

```
1  import numpy as np
2
3  # Convertir lista de Python a array NumPy
4  lista = [1, 2, 3, 4]
5  arr = np.array(lista)
6  print("Array:", arr)
7  print("Tipo:", type(arr))  # <class 'numpy.ndarray'>
8
9  # Array de ceros
10 zeros = np.zeros(5)
11 print("Zeros:", zeros)
12
13 # Array de unos
14 unos = np.ones((2,3))  # 2 filas x 3 columnas
15 print("Unos:\n", unos)
16
17 # Rango de valores
18 rango = np.arange(0, 10, 2)  # [0, 2, 4, 6, 8]
```


Primeros Pasos con NumPy $\ni$ Atributos de un Array

```
1 arr = np.array([[1, 2, 3],  
2                [4, 5, 6]])  
3 print("Shape:", arr.shape)    # (2, 3)  
4 print("Dim:", arr.ndim)       # 2  
5 print("Tipo de datos:", arr.dtype) # int64 (en sistemas Linux)  
6 print("Total de elementos:", arr.size) # 6
```

- `shape` = tupla de dimensiones.
- `dtype` = tipo de dato (`int32`, `float64`, etc.).
- `ndim` = número de ejes.

Manipulación de Arrays y Operaciones

Manipulación de Arrays y Operaciones $\ni$ Slicing e Indexación

```
1  arr = np.array([10, 20, 30, 40, 50])
2  print(arr[0])          # 10
3  print(arr[1:3])        # [20 30]
4
5  mat = np.array([[1, 2, 3],
6                  [4, 5, 6],
7                  [7, 8, 9]])
8  # Acceder a fila 1, columna 2
9  print(mat[1, 2])       # 6
10
11 # Slicing de filas y columnas
12 submat = mat[0:2, 1:3]  # Filas [0..1], Cols [1..2]
13 print(submat)
```

- **Ojo:** en NumPy, slicing crea **vistas**, no copias (con implicaciones en la modificación de datos).

Manipulación de Arrays y Operaciones $\ni$ Operaciones Elemento a Elemento (Vectorización)

```
1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 print("Suma:", x + y)    # [5  7  9]
5 print("Producto:", x * y) # [4 10 18]
6 print("Potencia:", x**2)  # [1 4 9]
```

- Sin bucles explícitos.
- El núcleo de NumPy optimiza estas operaciones.

- Mecanismo que NumPy utiliza para manejar operaciones entre arrays de diferentes formas.
- Ejemplo:

```
1 a = np.array([1, 2, 3])
2 b = 2
3 # b se "expande" para coincidir con la forma de a
4 res = a * b  # [2 4 6]
```

- También puede ocurrir con arreglos 2D y 1D, etc.

Manipulación de Arrays y Operaciones $\ni$ Ejemplo: Sumar Fila a una Matriz

```
1 mat = np.array([[1,2,3],
2                 [4,5,6],
3                 [7,8,9]])
4 fila = np.array([10, 20, 30])
5
6 # Broadcasting: se suma fila a cada fila de la matriz
7 res = mat + fila
8 print(res)
9 # [[11 22 33]
10 #  [14 25 36]
11 #  [17 28 39]]
```

- NumPy reconoce que `fila` es de 1D y la "extiende" sobre las filas.

Operaciones Numéricas y Práctica

```
1 arr = np.array([2, 4, 6, 8])
2
3 print("Suma total:", np.sum(arr))           # 20
4 print("Mínimo:", np.min(arr))               # 2
5 print("Máximo:", np.max(arr))               # 8
6 print("Promedio:", np.mean(arr))            # 5.0
7 print("Desv. Estándar:", np.std(arr))        # ~2.236
```

- También hay `np.median(arr)`, `np.var(arr)` y muchas más.
- Para arreglos 2D, se puede especificar `axis=0` o `axis=1`.

Operaciones Numéricas y Práctica $\ni$ Ejercicio 1:

Primer Contacto con NumPy



Enunciado

- Crear un arreglo 1D con los números del 1 al 10.
- Imprimir su **media**, **suma** y **producto** de todos los elementos.
- Reemplazar los valores ≤ 5 por 0, y los restantes por 1 (opcional: slicing lógico).
- Mostrar el arreglo resultante.

extbfConceptos: Arrays, operaciones vectorizadas, slicing lógico.

extbfFísica relevante: Manipulación de datos experimentales.

extbfSugerencia: Usa **`np.arange`** y operaciones de comparación (**`arr <= 5`**).

Operaciones Numéricas y Práctica $\ni$ Ejercicio 2:

Matriz y Broadcasting



Enunciado

- Crear una matriz 3x3 con $\{[1, 2, 3], [4, 5, 6], [7, 8, 9]\}$.
- Definir un **vector** $[10, 20, 30]$.
- Sumar dicho vector a cada fila de la matriz (broadcasting).
- Mostrar el resultado.

extbfConceptos: Manipulación 2D, broadcasting, suma de vectores.

extbfFísica relevante: Suma de señales o datos experimentales.

Operaciones Numéricas y Práctica $\ni$ Ejercicio 3:

Aproximación de π con Serie



Enunciado

- Usar `np.arange` para generar un rango de n valores.
- Calcular una aproximación de π usando la serie de Leibniz:

$$\pi \approx 4 \sum_{k=0}^n \frac{(-1)^k}{2k+1}$$

- Comparar el valor obtenido para distintos n .

extbfConceptos: Vectorización, sumas numéricas, aproximación de constantes.

extbfFísica relevante: Métodos numéricos en física y matemáticas.

extbfSugerencia: Emplear vectorización para computar $\frac{(-1)^k}{2k+1}$.

- Dividir la clase en **parejas o tríos**.
- Resolver los ejercicios en un **notebook de Colab**.
- Explorar ejemplos adicionales:
 - Generar **matrices aleatorias** (`np.random.rand()`) y calcular su determinante (**opcional, con `np.linalg.det`**).
 - Practicar slicing y reemplazo de submatrices.

- ¿Problemas de instalación o importación en Colab?
- ¿Dificultad para entender la forma (**shape**) de un arreglo?
- ¿Alguna confusión con slicing, vistas y copias?

Comparte tus dudas o experiencias, discutimos en conjunto.

Operaciones Numéricas y Práctica $\ni$ Solución 1 de Referencia:

Primer Contacto con NumPy

```
1  # Creamos el arreglo y calculamos estadísticas
2  import numpy as np
3  arr = np.arange(1, 11)  # [1..10]
4  print("Array:", arr)
5  print("Media:", np.mean(arr))    # 5.5
6  print("Suma:", np.sum(arr))      # 55
7  print("Producto:", np.prod(arr))# 3628800
8
9  # Reemplazo valores <=5 con 0, y >5 con 1
10 arr_mod = arr.copy()
11 arr_mod[arr_mod <= 5] = 0
12 arr_mod[arr_mod > 5] = 1
13 print("Array modificado:", arr_mod)
```

extbfDiscusión: El uso de slicing lógico permite modificar el arreglo de forma eficiente y sin bucles.

Operaciones Numéricas y Práctica $\ni$ Solución 2 de Referencia:

Matriz y Broadcasting

```
1  # Suma de vector a matriz usando broadcasting
2  import numpy as np
3  mat = np.array([[1,2,3],
4                  [4,5,6],
5                  [7,8,9]])
6  fila = np.array([10, 20, 30])
7  resultado = mat + fila # Broadcasting
8  print("Matriz resultante:\n", resultado)
9  # [[11 22 33]
10 #  [14 25 36]
11 #  [17 28 39]]
```

extbfDiscusión: Broadcasting permite sumar el vector a cada fila sin bucles explícitos.

Operaciones Numéricas y Práctica $\ni$ Solución 3 de Referencia: ✓

Aproximación de π

```
1  # Aproximación de pi usando la serie de Leibniz
2  import numpy as np
3
4  n = 1000000 # término superior
5  k = np.arange(n+1) # [0..n]
6
7  # (-1)^k = np.power(-1, k)
8  # denominador = (2*k + 1)
9  terminos = ((-1)**k) / (2*k + 1)
10 aprox_pi = 4 * np.sum(terminos)
11 print("Aproximación de pi con n =", n, ":", aprox_pi)
12
13 # Comparar con valor real
14 import math
15 print("Diferencia:", abs(math.pi - aprox_pi))
```

Discusión: La vectorización permite calcular sumas grandes de manera eficiente y rápida.

- **Verificación:** ¿Coincidió con tus resultados o surgieron discrepancias?
- **Rendimiento:** usar vectorización en lugar de bucles manuales.
- **Optimización:** manipulación y slicing evitan copias innecesarias.

Conclusiones

- **Aprendimos** los conceptos fundamentales de NumPy y su utilidad para cálculos numéricos.
- **Practicamos** la creación, manipulación y operaciones vectorizadas con arrays.
- **Exploramos** aplicaciones en problemas matemáticos y de simulación.
- **Desarrollamos** habilidades de análisis numérico y trabajo colaborativo.

Habilidad adquirida

Uso eficiente de arrays y operaciones vectorizadas para resolver problemas científicos.

- **Sesión siguiente:** Operaciones avanzadas con NumPy (reshaping, álgebra lineal) y visualización con Matplotlib.
- **Recomendación de estudio:** Repasa la documentación oficial de NumPy y experimenta con ejemplos propios.
- **Práctica sugerida:** Resuelve ejercicios adicionales con datos reales o simulados.

extbf;Nos vemos en la próxima sesión!